Monocular Depth Estimation

Monocular Depth Estimation
- 1、Enable Optimal Board Performance
  - 1.1、Enable Jetson Clocks
- 2、Program Launch
  - 2.1、Start the Camera and Manipulator Drivers
  - 2.2、Start the Monocular Depth Estimation Node Preview
- 3、Core Source Code Analysis
  - 3.1、Model Initialization and GPU CUDA Acceleration Backend
  - 3.2、Image Size Preprocessing and Tensor Conversion
  - 3.3、Depth Map Extraction and Pseudo-Color Rendering Display
  - 3.4、Automatic Safe Reset of the Manipulator Pose
- 4、Core Tunable Parameter Summary Table

This section demonstrates the application of **Monocular Depth Estimation** on robots based on neural networks. From a 2D image captured by a single ordinary RGB camera, a deep learning model is used to estimate in real time the physical depth distance of each region and object in the scene relative to the camera, and the relative depth is mapped to an intuitive pseudo-color heatmap as the output.

1、Enable Optimal Board Performance

1.1、Enable Jetson Clocks

Enabling Jetson Clocks ensures that all CPU and GPU cores run at maximum frequency, improving the model inference throughput of ONNX Runtime and the real-time frame rate: **(Not required on RDK and Raspberry Pi boards)**

```
sudo jetson_clocks
```

2、Program Launch

2.1、Start the Camera and Manipulator Drivers

Open the first terminal and enter the following command to start the base drivers of the camera and the manipulator:

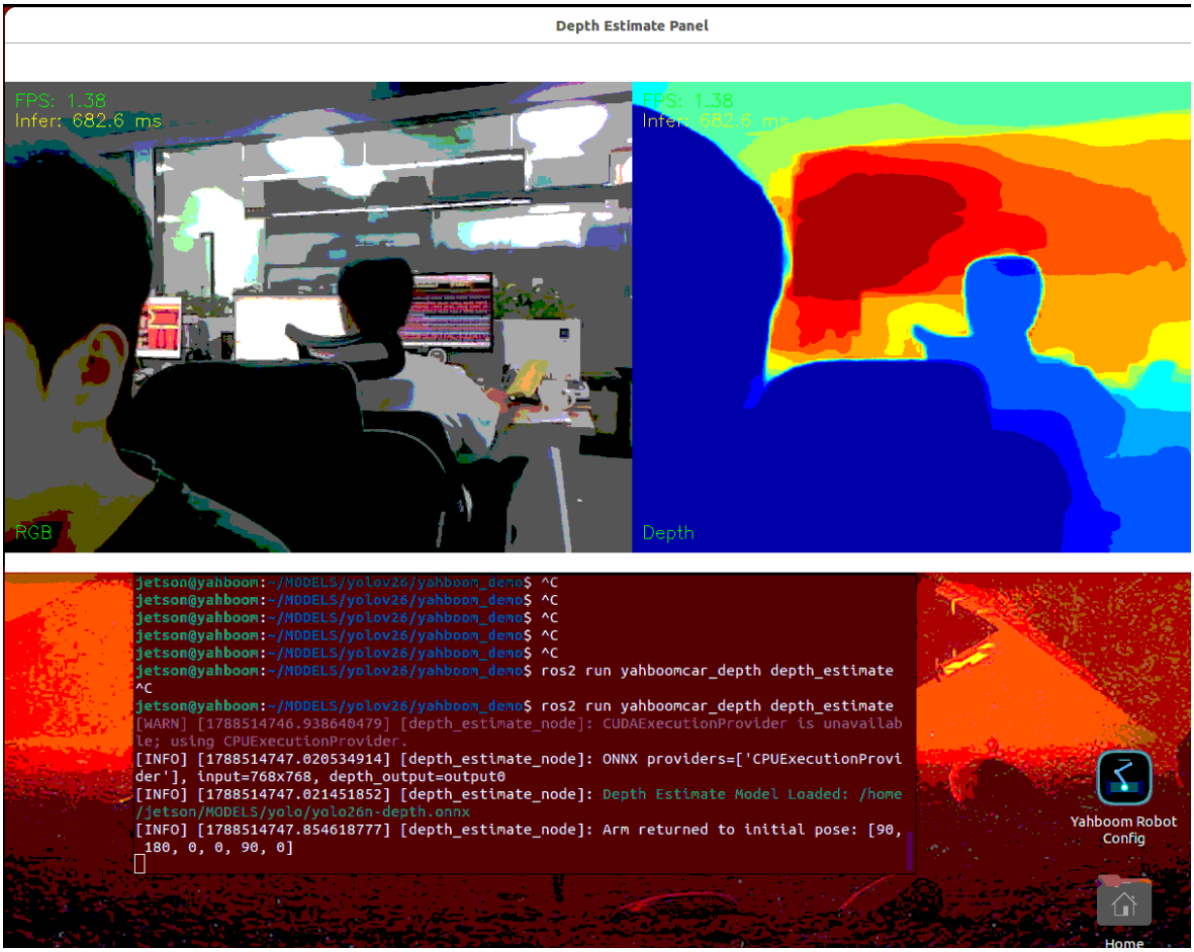
```
ros2 launch dofbot_pro_driver A1pro_camera_arm.launch.py
```

> **Note**: The program automatically adapts the input image Topic according to the system environment variable `CAMERA_TYPE`:
>
> - If using the **Nuwa depth camera** (default): automatically subscribes to `/ascamera_hp60c/camera_publisher/rgb0/image`;
> - If using a **USB camera**: automatically subscribes to `/usb_cam/image_raw`.

2.2、Start the Monocular Depth Estimation Node

Open the second terminal and enter the following command to start the main depth estimation program:

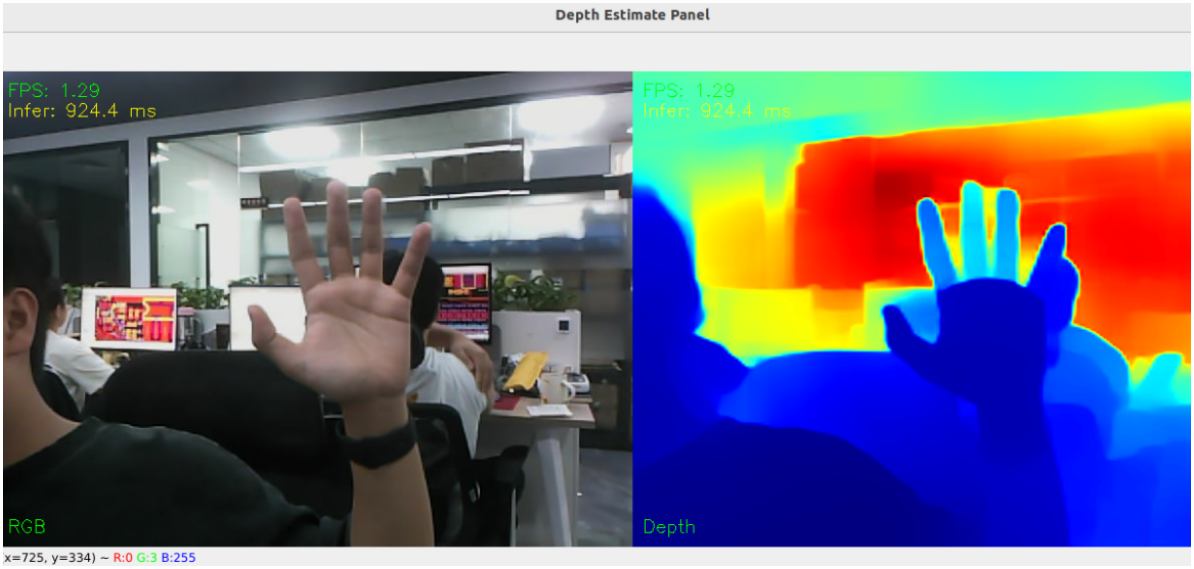
```
ros2 run yahboomcar_depth depth_estimate
```



After the program starts, it automatically performs the following operations:

1. **Automatic reset of the safe manipulator pose**: publishes the initial pose `(90, 180, 0, 0, 90, 0)` to `arm6_joints` to return the manipulator to its home position, preventing the manipulator structure from blocking the camera view;
2. **Loads the ONNX model and detects hardware acceleration**: loads the `~/MODELS/yolo/yolo26n-depth.onnx` model by default and preferentially enables the GPU CUDA execution provider (`CUDAExecutionProvider`) for acceleration;
3. **Pops up a dual-frame real-time window**: a window titled `Depth Estimate Panel` is shown, with the original RGB color frame on the left and the monocular depth estimation result on the right.

Preview




```
Frame Display Description:
- Left frame 【RGB】: real-time color camera frame, showing the current frame rate
(FPS) and the per-frame inference time (Infer ms) in the upper-left corner;
- Right frame 【Depth】: the dense relative depth pseudo-color map generated by
monocular estimation (default JET mode; red/warm colors indicate distant objects,
while cold/blue colors indicate nearby background).
```

3、Core Source Code Analysis

The source file is located at:

```
~/yahboomcar_ros2_ws/yahboomcar_ws/src/yahboomcar_depth/yahboomcar_depth/Yolov26
/depth_estimate.py
```

3.1、Model Initialization and GPU CUDA Acceleration Backend

The program uses ONNX Runtime as the inference backend and automatically detects the runtime environment of the current system during initialization; if CUDA is supported, GPU hardware acceleration is preferentially enabled:

```
def init_depth_model(self):
    '''Load the monocular depth estimation ONNX model and cache the input/output
metadata'''
    if not os.path.isfile(self.model_path):
        raise FileNotFoundError(f'Depth ONNX model not found:
{self.model_path}.')

    available_providers = ort.get_available_providers()
    providers = ['CPUExecutionProvider']
    # Preferentially detect and mount the CUDA GPU inference acceleration engine
    if self.use_cuda and 'CUDAExecutionProvider' in available_providers:
        providers.insert(0, 'CUDAExecutionProvider')
    elif self.use_cuda:
        self.get_logger().warning('CUDAExecutionProvider is unavailable; using
CPUExecutionProvider.')
```

```
    self.ort_session = ort.InferenceSession(self.model_path,
providers=providers)
    input_meta = self.ort_session.get_inputs()[0]
    self.input_name = input_meta.name
    self.input_height = self.static_dimension(input_meta.shape[2], 640)
    self.input_width = self.static_dimension(input_meta.shape[3], 640)
    self.input_dtype = np.float16 if input_meta.type == 'tensor(float16)' else
np.float32
```

3.2、Image Size Preprocessing and Tensor Conversion

To meet the input requirements of the neural network model, the original BGR color image is first converted to RGB format, resized to the model's fixed resolution (e.g., 640×640), normalized to `[0, 1]`, and converted into a batch tensor of shape `(1, 3, H, W)`:

```
def prepare_input(self, rgb_image):
    input_image = cv2.cvtColor(rgb_image, cv2.COLOR_BGR2RGB)
    input_image = cv2.resize(input_image, (self.input_width, self.input_height))
    input_tensor = input_image.astype(self.input_dtype) /
self.input_dtype(255.0)
    input_tensor = np.transpose(input_tensor, (2, 0, 1))[None, ...]
    return np.ascontiguousarray(input_tensor)
```

3.3、Depth Map Extraction and Pseudo-Color Rendering Display

The single-channel depth map output by the model is mapped to a 0~255 grayscale space via `cv2.normalize` and then rendered into an intuitive pseudo-color image through an OpenCV color palette. The depth map can also be published as a ROS topic for downstream nodes:

```
def get_RGBImageCallBack(self, rgb_msg):
    rgb_image = self.rgb_bridge.imgmsg_to_cv2(rgb_msg, desired_encoding='bgr8')
    infer_start = time.time()
    try:
        input_tensor = self.prepare_input(rgb_image)
        output = self.ort_session.run([self.depth_output_name],
{self.input_name: input_tensor})[0]
        depth_map = self.extract_depth_map(output)
    except Exception as error:
        ...
        return

    # 1. Normalize to 0~255 and convert to an 8-bit integer
    depth_vis = cv2.normalize(depth_map, None, 0, 255,
cv2.NORM_MINMAX).astype(np.uint8)
    # 2. Pseudo-color rendering (default COLORMAP_JET)
    depth_vis = cv2.applyColorMap(depth_vis, self.colormap)
    if depth_vis.shape[:2] != rgb_image.shape[:2]:
        depth_vis = cv2.resize(depth_vis, (rgb_image.shape[1],
rgb_image.shape[0]))

    # 3. Optional: publish the depth map to a ROS topic
    if self.depth_pub is not None:
        depth_msg = self.rgb_bridge.cv2_to_imgmsg(
```

```
            cv2.normalize(depth_map, None, 0, 255,
cv2.NORM_MINMAX).astype(np.uint8),
            encoding='mono8',
        )
        depth_msg.header = rgb_msg.header
        self.depth_pub.publish(depth_msg)

    # 4. Horizontally concatenate the original frame and the depth frame and
display them in real time
    combined_frame = cv2.hconcat([rgb_image, depth_vis])
    cv2.imshow("Depth Estimate Panel", combined_frame)
    cv2.waitKey(1)
```

3.4、Automatic Safe Reset of the Manipulator Pose

After the node starts, it sends servo control commands to `arm6_joints` through a timer to move the 6-DOF manipulator to the preset safe reset pose, ensuring that the manipulator does not block the camera:

```
def reset_arm_pose(self):
    '''Move the 6 servos of the manipulator back to the reset initial pose to
avoid blocking the forward view'''
    msg = ArmJoints()
    msg.joint1, msg.joint2, msg.joint3 = self.init_pose[:3]
    msg.joint4, msg.joint5, msg.joint6 = self.init_pose[3:]
    msg.time = ARM_MOVE_TIME_MS
    self.arm_pose_pub.publish(msg)
    self.arm_init_timer.cancel()
    self.get_logger().info(f"Arm returned to initial pose:
{list(self.init_pose)}")
```

4、Core Tunable Parameter Summary Table

Parameter	Default Value	Description
`model_path`	`~/MODELS/yolo/yolo26n-depth.onnx`	Absolute path of the depth estimation ONNX model file
`image_topic`	Auto-detected (USB or Nuwa)	ROS 2 topic name of the input color image
`use_cuda`	`True`	Whether to preferentially enable GPU CUDA for model inference acceleration
`publish_depth`	`False`	Whether to publish the generated single-channel depth map to the `/depth_image` topic
`depth_colormap`	`JET`	Pseudo-color mapping table of the depth map; options: `JET`, `TURBO`, `INFERNO`, `MAGMA`
`init_pose`	`[90, 180, 0, 0, 90, 0]`	The 6 joint angles (in degrees) for automatic reset of the manipulator at startup